Machine Learning Exercise 1: Classification

Group 11

András Bonifác Kónya	Student ID: 01502933
Branimir Raguž	Student ID: 12123474
Thummanoon Kunanuntakij	Student ID: 12122522

---

1. Overview

In this report, we explore the process of applying machine learning experiments to various datasets in order to do classification. For this, we used 3 kinds of machine learning algorithms and applied them to 4 different datasets of various characteristics, two of which were assigned and the others were chosen by us.

2. Experiment Process

We went through each dataset with a similar process using the following step

2.1 First we describe some interesting characteristics of the dataset.
2.2 We splitted the data into 3 separate sets. training data, evaluation data and testing data.
2.3 Then, we did data exploration and saw if we needed any pre-processing.
2.4 After that, we calculate 2 baseline methods. One is a naive method which predicts only the most occurrence class and the other is to fit a model using the default parameters of the algorithm.
2.5 And then, we experimented with various parameters of the model to find an optimal set of parameters which could improve the performance of the default parameters.
2.6 Finally, we fitted the testing data to see the final performance and discussed which set of parameters could be the best one.

3. Data Preprocessing Methods

1-Hot Encoding

A method to encode a column of categorical data into a boolean column per category. The advantage of this is that each column can influence the model on its own and, unlike label encoding, it doesn’t imply any relative size. The only problem that it may have is that if the categories in the training data are not an exhaustive list, the new category will be ignored completely.

Scaling

Some Machine Learning algorithms such as K-Nearest Neighbour and Regularized Regression require the input data to be in a similar scale so that it can account for the impact of each variable equally. There are many methods that we can use to scale the data. The popular ones are the following.

- Min-Max: scale each value of a parameter by the range of the parameter.
- Standard: scale each value of a parameter by the standard deviation of the parameter, so it has zero mean and unit variance.
- Max Absolute: scale each value of a parameter by its maximum value.
- Normalizer: scale each observation to have a unit length.

PCA

When there are too many parameters or dimensions in our data, we could do dimension reduction by using Principal Component Analysis. This method merges all of the parameters together and finds the specified number of dimensions with the highest variance.

4. Machine Learning Algorithm

Regularized Logistic Regression

Implementation used:
https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

Overview

This algorithm tries to learn a model by finding a suitable linear plane that is able to separate each class of the target variable from another. When it is used for a multiclass problem, it is usually trying to do the 1 vs the rest strategy which means it would pick a class and then try to find the best model to separate this class from the other classes.

Assumption

- When making a regularized model, the parameters should be in the same scale so that the algorithm could penalize the size of each parameter correctly.
- No multicollinearity amongst the parameters.
- No influential outlier.
- Linearity in the logit (log of odd) for continuous variables.

Parameter of Interest

penalty: {‘l1’, ‘l2’, ‘elasticnet’, ‘none’}
This parameter specifies how the algorithm should panaltize the size of the coefficient, L1, L2 or both (elasticnet). If it is ‘none’, a normal least square regression is performed.

solver: {‘newton-cg’, ‘lbfgs’, ‘liblinear’, ‘sag’, ‘saga’}

This parameter specifies which optimization algorithm is used. Some algorithms are able to fit only some type of penalty as list in the following.

Type of penalty available for each solver

‘newton-cg’	[‘l2’, ‘none’]
‘lbfgs’	[‘l2’, ‘none’]
‘liblinear’	[‘l1’, ‘l2’]
‘sag’	[‘l2’, ‘none’]
‘saga’	[‘elasticnet’, ‘l1’, ‘l2’, ‘none’]

C: *float, default=1.0*
Inverse of regularization strength; must be a positive float. Smaller values specify stronger regularization.

K-Nearest Neighbor

Implementation used:
https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html

Overview

K-Nearest Neighbor classified an input data by using a simple majority vote of the nearest neighbors of the point.

Assumption

Parameters should be in a similar scale so that the size of each dimension impacts the distance in the same scale.

Parameter of Interest

n_neighbors int, default=5
Number of neighbors that are used for voting.

p int, default=2
Type of distance used for calculating the neighbour.

p = 1	Manhattan Distance (L1)
p = 2	Euclidean Distance (L2)
For arbitrary p	Minkowski Distance (Lp)

Random Forest

Implementation used:
https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

Overview

This model fits a number of decision tree classifiers on various sub-samples of the dataset and uses averaging to improve the predictive accuracy and control over-fitting.

Assumption

Random Forest doesn't have any assumption on the input data.

Parameter of Interest

n_estimators int, default=100
Number of decision tree that is used.

criterion {"gini", "entropy"}, default="gini"
The function to measure the quality of a split.

max_depth int, default=None
The maximum depth of the tree.

min_samples_split int or float, default=2
The minimum number of samples required to split an internal node:

min_samples_leaf int or float, default=1
The minimum number of samples required to be at a leaf node.

min_impurity_decrease float, default= 0.0
A node will be split if this split induces a decrease of the impurity greater than or equal to this value.

max_features {"auto", "sqrt", "log2"}, int or float, default="auto"
The number of features to consider when looking for the best split.

5. Evaluation Metrics

For classification problems, there are various matrices that can be used to evaluate the models.

Accuracy

This is the most straightforward method. It is just the percentage of correctly classified data to the total number of data. The downside of this is that it may be misleading for some dataset that the number of data points in each class is largely different, for example when trying to predict if someone would catch a rare disease. With only 1% of the total data point being positive, simply predicting that a patient has not been infected with the disease can reach 99% accuracy.

Because of this, we have the following metrics which could be a better indicator of the performance of a model specifically for the binary classification problems.

Confusion Matrix

The matrix shows each type of classification outcome. Each value of the cells are used to compute more metrics below.

	Actual Positive	Actual Negative
Predict Positive	True Positive (TP)	False Positive (FP)
Predict Negative	False Negative (FN)	True Negative (TN)

Precision

$$\frac{TP}{TP+FP}$$

The rate of correctly classifying positive cases from all positive predictions. It is used when the cost of False Positives is high.

Recall / Sensitivity / True Positive Rate (TPR)

$$\frac{TP}{TP+FN}$$

The rate of correctly classifying positive cases from all positive cases. It is used when the cost of False Negatives is high.

F1 Score

$$\frac{2 \cdot Precision \cdot Recall}{Precision + Recall}$$

The harmonic mean of the model's precision and recall. It combine Precision and Recall in the same metrics

For this report, we use mainly 2 metrics, accuracy and F1 score.

6.1 Dataset 1: Glass Identification

Link: https://archive.ics.uci.edu/ml/datasets/glass+identification

Description

This is a classic dataset from UCI Machine Learning Repository. The goal of this data set is to predict the type of the glass using the given features. We pick this dataset because it is a small dataset with only 214 Observations and also it is a multiclass classification problem. We are interested in how to deal with such a small dataset and to see if predicting multiclass classification is different from predicting the binary one.

Data Exploration & Preprocessing

The quality of this dataset is quite high. All of the features are in numeric. We could see that there are no obvious outliers that we need to deal with and also the distribution of most features somewhat resembles the bell curve. But because the scale of the data is quite different, performing data scaling could be a benefit.

Training / Validation / Testing Set

We reserved 15% of the data for the testing set. The rest is for modeling. Then we split the data again and use 20% of the modelling data for validation. The rest is for training.

Evaluation Metric

We would use mainly Accuracy when performing any parameter tuning. But we also include F1 Macro to get further insight.

During the parameter optimization we train our model using 3-fold cross validation using accuracy as a metric. But we calculate F1 Macro for the validation set and the testing set.

Model 1: Logistic Regression

Parameter Tuning

First we use the default parameter of the model from Scikit-learn. Then we will try to improve the model by picking the best performing parameter one by one. We found that using `None` and `StandardScaler()` were performing at a similar level. We pick both of them at this step because we know that the Standard Scaler could be a benefit to our regularized regression model. At this step, we try all eligible combinations of solver/penalty. To see which one gets the best performance. For the no scaler, the best solver is `saga` with no penalty. And for the standard scaler, the best solver is `liblinear` with L2 penalty.

At this step, we try to find the best regularized parameter, C, for the model with L2 penalty. The optimal value for this parameter is C=10.

Evaluation

We found that the standard scaler version is the best model for this algorithm. Our chosen parameters are solver='liblinear', penalty='l2' and C=10.

Accuracy of CV training set Logistic Regression(default)): 0.6111
Accuracy of Validation set Logistic Regression(default)): 0.4865
Macro F1 Score of Validation set Logistic Regression(default)): 0.348

Optimal parameter training set score (no scaler) cv: 0.625
Optimal parameter set score (no scaler): 0.4595
F1 Score of Validation set Logistic Regression (no scaler)): 0.3477

Optimal parameter training set score (standard scaler) cv: 0.6458
Optimal parameter set score (standard scaler) 0.5135
F1 Score of Validation set Logistic Regression (standard scaler)): 0.3592

Model 2: K-Nearest Neighbour

Parameter Tuning

We started with the default model and found that the standard scaler performs significantly better than others. Then, we use the standardized data to tune n_neighboar and p. We found that using p=1 and 3 neighbors perform the best.

Evaluation

We could see that our model can beat the default model in terms of accuracy but we could all see that the F1 score macro is the same. Our chosen parameters are p=1 and n_neighbors=3.

Accuracy of CV training set KNN(default)): 0.6319
Accuracy of Validation set KNN(default)): 0.6216
F1 Score of Validation set KNN(default)): 0.3560

Optimal parameter training set score KNN (standard scaler): 0.7361
Optimal parameter set score KNN (standard scaler) validation: 0.6756
F1 Score of Validation set KNN (standard scaler)) validation: 0.3560

Model 3: Random Forest

Parameter Tuning

Again, we started with the default model and optimized each parameter one by one. We found that almost all scalers perform at a similar level with `None`. So we don't apply any scaler at all. Then, we found that using `criterion='entropy' is significantly better. Finally we optimized `n_estimators` and `max_depth` and got n_estimator=500 and max_depth=5 as the best parameters

Evaluation

We found that the optimal forest did a better job at predicting the validation set than the default model. Our chosen parameter are n_estimators=3000, max_depth=10, criterion='entropy'

Accuracy of CV training set Random Forest(default)): 0.7013
Accuracy of Validation set Random Forest(default)): 0.7027
F1 Score of Validation set Random Forest(default)): 0.5200

Accuracy of CV training set Random Forest(optimal)): 0.7013
Accuracy of Validation set Random Forest(optimal)): 0.7837
F1 Score of Validation set Random Forest(optimal)): 0.6789

Discussion

We can see that the accuracy of each method doesn't differ very much but the F1 scores are significantly different. Here we can clearly choose the random forest as a final model.

The final parameter are criterion='entropy', n_estimator=3000 and max_depth=10

	Test Accuracy	Test F1 Macro
Naive Baseline	0.3333	0.0833
Logistic Regression	0.6969	0.5412
K-Nearest Neighbors	0.6666	0.6673
Random Forest	0.7878	0.7402

6.2 Dataset 2: Telecom Churn

Link: https://www.kaggle.com/mnassrib/telecom-churn-datasets

Description

This dataset contains data about customer telephone activities. The goal is to predict where the customer will cancel telephone subscription or not in the future so the company could create a retention program for the churning group.

The dataset has a practical application in telecommunication business. Using the dataset allows us to learn to apply our machine learning algorithms to the real world. It is labelled with a full score in usability in Kaggle so we can ensure that the data quality is high.

Data Exploration & Preprocessing

For the telecom churn dataset, we do some data cleaning by encoding ‘Yes’/’No’ columns into binary columns to make it easier for our algorithms. Then, we do One-Hot Encoding for the ‘Area Code’. Finally, we decided to drop the ‘State’ column out because if you encode this columns we will add many more parameters to our data.

The data contains no significance outlier and also they look mostly normal so we don’t need to do any necessary transformation. But the scale of the data still differs quite a bit so we will try to do some scaling for each model.

Training / Validation / Testing Set

The original data already split the testing data for us, so we kept the original testing set. Then, we split 20% of the training data for the validation set.

Evaluation Metric

We would use mainly Accuracy when performing any parameter tuning. But we also include F1 Macro to get further insight.

During the parameter optimization we train our model using 3-fold cross validation using accuracy as a metric. But we calculate F1 Macro for the validation set and the testing set.

Model 1: Logistic Regression

Parameter Tuning

For Logistic Regression, we start with the default parameter and try to find the right scaling. We found that without scaling and standard scaling perform at a similar level. Thus, we try to use both for optimizing other parameters. Then we optimize for solver and penalty and found that the model with no scaling is best with solver=’lbfgs’ and penalty=’l2’, while the model with standard scaler is best with solver=’newton-cg’ and penalty=’l2’. Lastly we find the best regularized parameter, C. We got C=0.1 for the no-scaling model and C=0.3 for the standard scaling model.

Evaluation

We could see that our non-scaling case provides the best performance. Our chosen parameters are `solver='lbfgs'`, `penalty='l2'`, and `C=0.1`

Accuracy of CV training set Logistic Regression(default)): 0.8616
Accuracy of Validation set Logistic Regression(default)): 0.8614
F1 Score of Validation set Logistic Regression(default)): 0.2884

optimal parameter training set score (no scaler) cv: 0.8625
optimal parameter set score (no scaler) 0.8651

F1 Score of Validation set Logistic Regression (no scaler)): 0.2941

optimal parameter training set score (standard scaler) cv: 0.8616
optimal parameter set score (standard scaler) 0.8614
F1 Score of Validation set Logistic Regression (standard scaler)): 0.2884

Model 2: K-Nearest Neighbour

Parameter Tuning

For this model we found that the best scaling is the standard scaler and Min-Max scaler. So we use both to find the best `p` and `n_neighbors` and we found that for the standard scaler model, we got p=1 and n_neiggbors=3 and for the Min Max scaler we got p=1 and neighbors=1 as well.

Evaluation

The min-max scaler KNN with parameter `p=1` and `n_neighbors=3` performs the best for the validation set.

Accuracy of CV training set KNN(default)): 0.8742
Accuracy of Validation set KNN(default)): 0.8726
F1 Score of Validation set KNN(default)): 0.3928

Optimal parameter training set score KNN (standard scaler): 0.8907
Optimal parameter set score KNN (standard scaler) validation: 0.883
F1 Score of Validation set KNN (standard scaler)) validation: 0.3928

optimal parameter training set score (min max) cv: 0.8832
optimal parameter set score (min max) validation: 0.8876
F1 Score of Validation set KNN (standard scaler)) validation: 0.4545

Model 3: Random Forest

Parameter Tuning

As with the previous dataset, this algorithm needs no scaling, so we moved to optimize for the criterion and found that `entropy` outperform `gini`. Then for the `n_estimators` and `max_depth` we

Evaluation

We found that the optimal forest did a better job at predicting the validation set than the default model.

Accuracy of CV training set Random Forest(default)): 0.9455

Accuracy of Validation set Random Forest(default)): 0.9550
F1 Score of Validation set Random Forest(default)): 0.8153

Accuracy of CV training set Random Forest(optimal)): 0.9446
Accuracy of Validation set Random Forest(optimal)): 0.9606
F1 Score of Validation set Random Forest(optimal)): 0.8396

Discussion

We can see that the accuracy of each method doesn't differ very much but the F1 scores are significantly different. Here we can clearly choose the random forest as a final model.
The final parameter are criterion='entropy', n_estimator=500 and max_depth=5

	Test Accuracy	Test F1
Naive Baseline	0.8575	0.0
Logistic Regression	0.8590	0.2539
K-Nearest Neighbors	0.8950	0.5138
Random Forest	0.9580	0.8292

6.3 Dataset 3: Purchase (Kaggle Competition)

Link: https://www.kaggle.com/c/184702-tu-ml-ws-21-location

Description

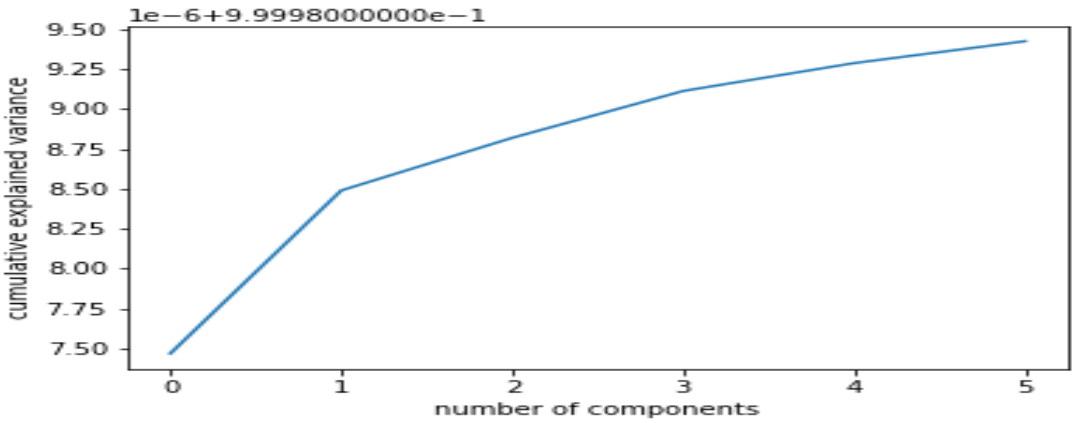
The purchase dataset contains information about 10000 customers and 600 binary attributes (product types), showing if a customer purchased the corresponding product or not. There are 100 classes in the target attribute. Each class represents a group of customers with similar purchase behaviour. The classification task is to predict the purchase behaviour of customers.

Data Exploration & Preprocessing

The training data sets dimensions are 10000 rows x 602 columns. Since the entire data set was a big set of 0s and 1s, we did not have much interesting to conclude. I tried visualizing the correlation between variables but as there are 600 attributes the plot wasn’t of much value. The number of dimensions was too big, so it would take a lot of time to train and optimize the model. We concluded that the best approach is to use a dimension reduction technique in the preprocessing part.

There were no missing values, so we did not have to deal with that issue. There were no outliers as well, as the data again was cleaned. There was no categorical data to encode. Scaling was not required, and the train/test split was provided. We split the df_train file into training and test data to test out our models before the submission. The biggest thing we did in preprocessing was to apply different dimensionality reduction techniques to see how the data would change, and the models would improve. Applying different models to the data before dimensionality reduction produced terrible results. After researching how to scale a data full of 1s and 0s, and not finding anything concrete we tried Principal Component Analysis and Singular Value Decomposition.

Principal component analysis variance plot



The Principal component analysis variance plot tells us how much variance is captured when considering different n values of components. 6 components will capture 97% of the variance, but we choose 10 as the final value because the accuracy scores were slightly better.

Model 1: Logistic Regression

Parameter Tuning

There are not many parameters to tune in the Logistic Regression method. At first the method didn't work because the default solver parameter wasn't appropriate for the problem. We changed the solver to liblinear. We only tuned one more thing in the logistic regression and that is the penalty value. We changed it from the default value of l2 (regularization tries to estimate the mean of the data to avoid overfitting) to l1(regularization tries to estimate the median of the data to avoid overfitting), and we saw a slight increase in accuracy.

Evaluation

The logistic regression approach deemed solid results. The best accuracy we managed to score was 0.6. We guess the problem was that the data was not clearly linearly separable, and that confused the algorithm.

Model 2: K-Nearest Neighbour

Parameter Tuning

Similar to the Logistic Regression, there were not many parameters to tune that would provide an improvement in the results. The first default model produced terrible accuracy results of 0.0178. The problem was in the distance metric used. I tried almost all distance metrics and they all produced similarly bad results. The results improve significantly when we set the distance metric to correlation. We then went on to experiment with different values for k_neighbors. We tried the values [(1-21) incr. By 2], and the best value was 19. The best KNN model in the end is with the n_neighbors value of 19 and the correlation used as the metric.

Evaluation

The K nearest neighbors algorithm provided the worst results on the purchase data set.

Model 3: Random Forest

Parameter Tuning

There were a lot of parameters to tune for Random Forest. It is the algorithm that provided us the best results in the end. We first tuned the n_estimators parameter. Knowing that the higher number of estimators provide better results, we looked for a value that captures almost the best accuracy, but isn't too big of a number. We decided on 300 as the value. We then tested a different function to measure the quality of a split (criterion='entropy'), but we stayed with the default value of 'gini'. We then tuned the max_depth parameter. The default value of none is probably the best because that way the nodes are expanded until all the leaves are pure or until all leaves contain less than min_samples_split samples. After trying different values, the model didn't improve. We then tuned the min_samples_split parameter and concluded that the default value was the best one. We also tried to tune the max_features parameter. The best value of considering the number of features when looking for the best split is 'none', the default one. The final best model was with n_estimators set to 300, and other parameters set to default.

Evaluation

Random forest approach deemed the best results. Because of its independence on any set of features it can generalize over data in a better way.

Discussion

The purchase data set was an interesting problem to solve. As the data set was already cleaned, and scaled, the information in the description was everything we knew about it. There were no patterns or points of interest to analyze. Most of the process was tuning the parameters. The principal component analysis was the step that had the biggest impact on the outcome. It combined with the tuned Random Forest Classifier provided a decent result. The logistic regression method worked well with the purchase dataset. K nearest neighbors algorithm simply failed.

Optimal models

Evaluation metric	K nearest neighbor	Logistic regression	Random forest
Accuracy	0.3789	0.6053	0.6753
F1 Score	0.3207	0.5466	0.6323

6.4 Dataset 4: Breast Cancer (Kaggle Competition)

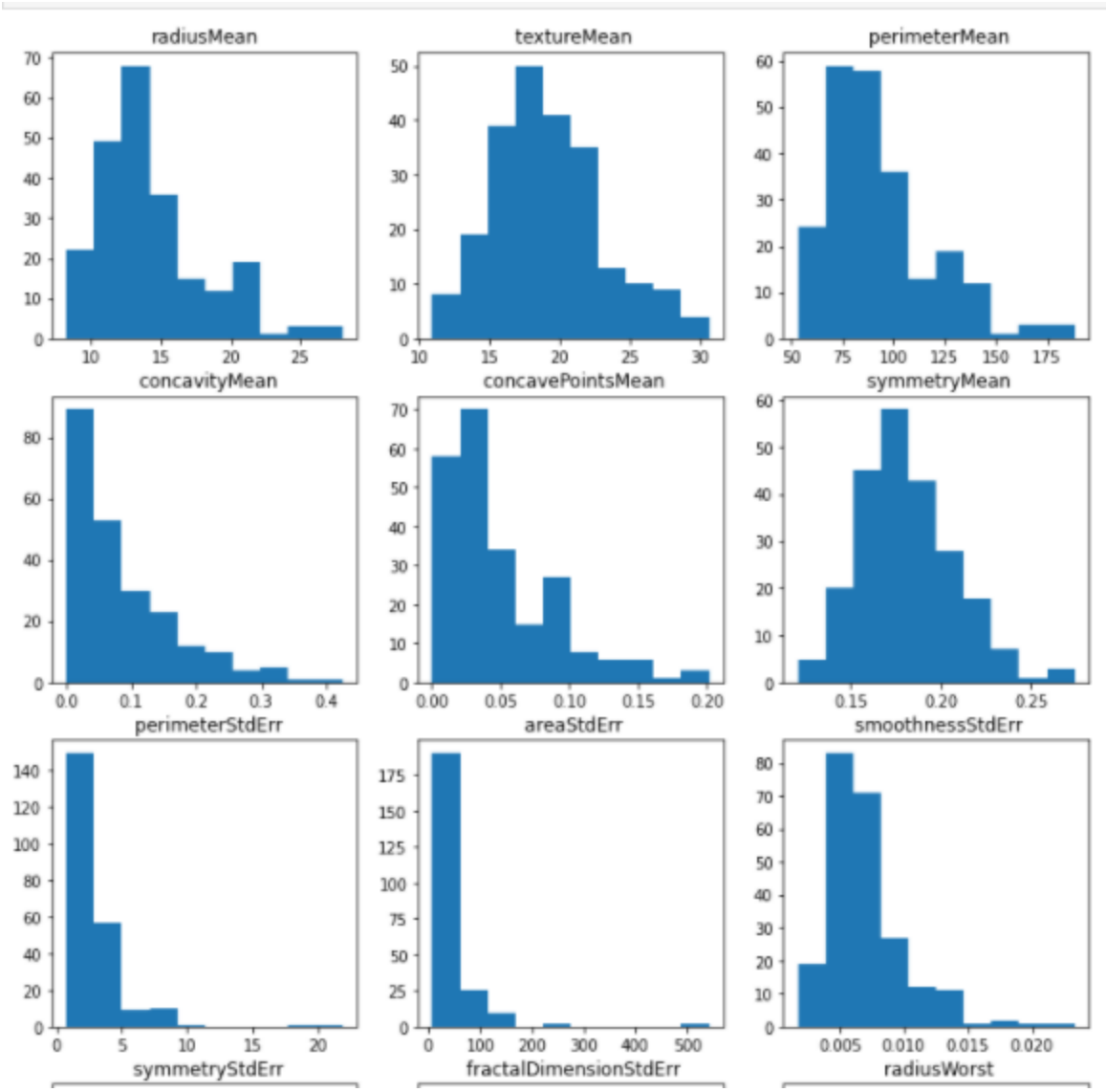
Link: https://www.kaggle.com/c/184702-tu-ml-ws-21-breast-cancer

Description

Our 4th dataset is about patients with breast cancer, and to predict if a patient has this disease or not. The origin of this dataset can be traced back to a kaggle competition with 3 separate csv files. 1 file for training, 1 file for testing, and 1 file for evaluation. The data has 32 columns and different patients are classified based on their ID and the class is responsible for storing the result to see if the patient has a real illness. The other 30 columns refer to the parameters, measurements, and observations performed on the patient.

Data Exploration & Preprocessing

Data preprocessing begins primarily with handling missing data or NULL values. We must first review whether our data contain such values at all. Neither NULL values nor missing data can be found in the breast cancer dataset, which facilitates the preprocessing phase.
This process is followed by searching for Outliers that could not be found in the dataset based on that figure:



Seeing next on the figure the following about **feature scaling**: the scale of each variables are differ, so it could be worthwhile if we do some standardization to our data. But each machine learning algorithms require different criteria for its input data so we would do the feature transformation later during the analysis of each model.

Then evaluation metrics follow, which is the following:
Accuracy: This is the most common and stright forward one. It tells us the percentage of the number of correct prediction.
F1: This metric combines precision and recall. It is a better indicator of the performance of our model when the output class is imbalance like in our case where the churning is only 17%.
In conclusion, the Naive Baseline Model, where we can see that the accuracy is quite high because the number of churning samples are low.

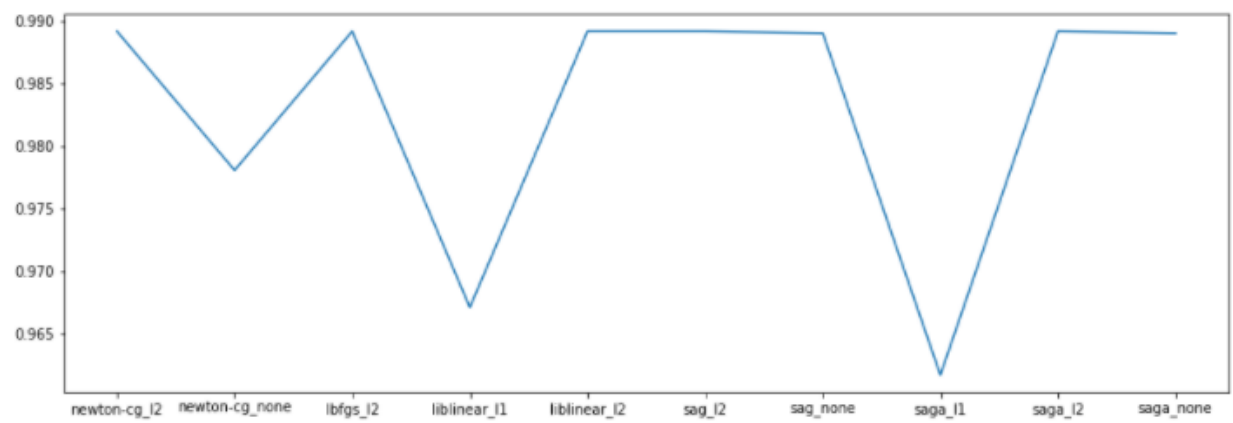
Model 1: Logistic Regression

In the case of logistic regression, we first use the basic model to calculate the prediction. The default model is used to compare with our final model whether we can improve its performance or not, which has a surprisingly high value: 0.9453.

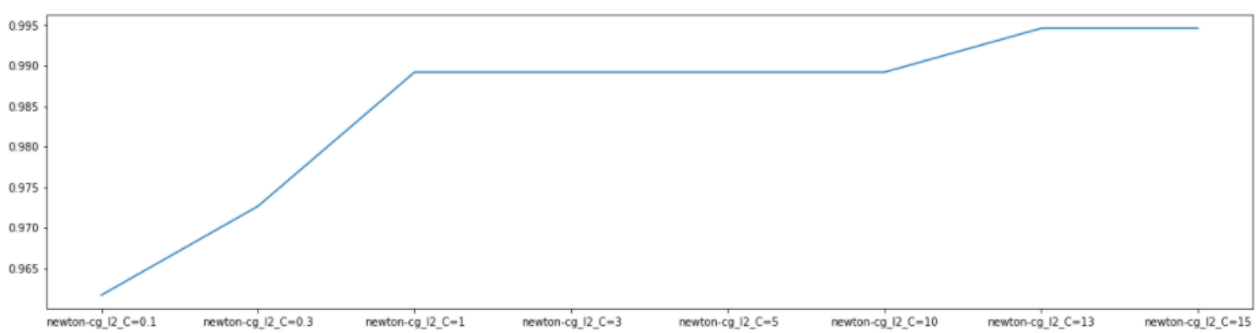
Parameter Tuning

We primarily use Parameter scaling for Parameter tuning to find out which transformation can produce the best cross-validation value for us. Based on your feedback: we could see that no scaler and using standardScaler () provide close performance. We picked these two to transform our dataset because we know that the scale of the parameter could affect the performance of the regularized model.

We then use the Solver / Penalty Tuning Parameter, which can determine the maximum accuracy for the standard scaler, which in our case is 0.9891, i.e. almost 99%. The best model is newton-cg_l2, which can be seen on the plot:



For tuning parameter C, we could see that solver = newton-cg, penalty = l2 and C = 13 perform the best:

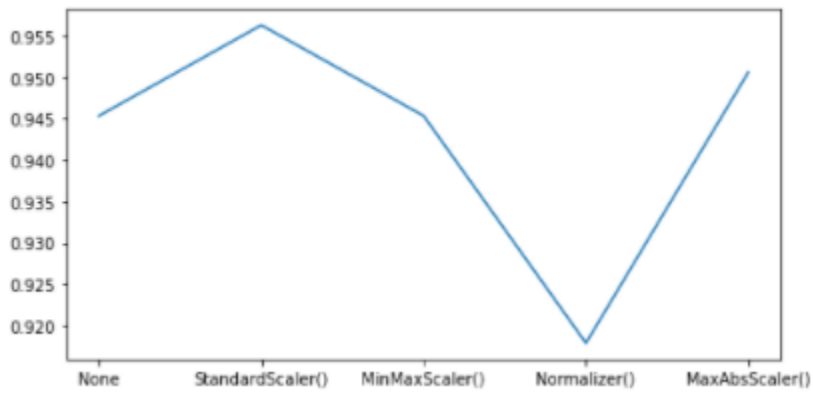


Evaluation

The logistic regression performed surprisingly well, it was also able to leave the random forest algorithm behind with the following values:
optimal parameter training set score (standard scaler) cv: 0.9945
optimal parameter set score (standard scaler) 0.9565
F1 Score of Validation set Logistic Regression (standard scaler)): 0.9375

Model 2: K-Nearest Neighbor

Like logistic regression, K-Nearest Neighbor also starts with a default model and has the following parameter scaling:



Parameter Tuning

For the Tuning Parameter n_neighbors and p, the following results were obtained:

	setting	accuracy
14	p=2_n_neighbors=5	0.956306
3	p=1_n_neighbors=4	0.956306
4	p=1_n_neighbors=5	0.956306
28	p=3_n_neighbors=9	0.956156
26	p=3_n_neighbors=7	0.956156

Evaluation

For K-Nearest Neighbor Optimal Model we got the following parameters:
Optimal parameter training set score KNN (standard scaler): 0.9563
Optimal parameter set score KNN (standard scaler) validation: 0.9130
F1 Score of Validation set KNN (standard scaler)) validation: 0.9090

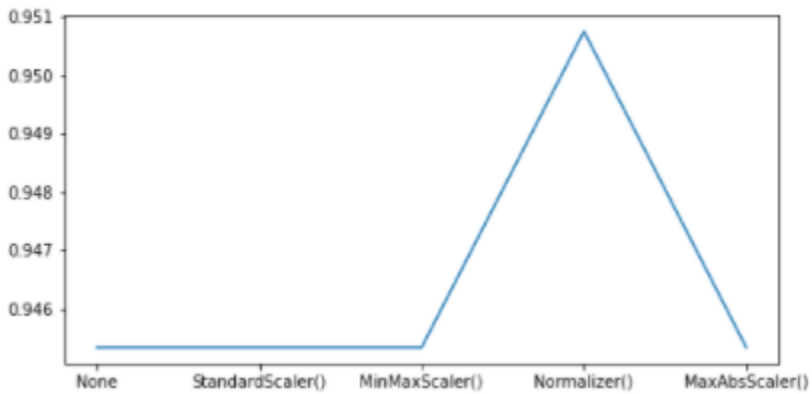
The standard scaler KNN with parameter p=2 and n_neighbors=5 performs the best for the training set. But it performs similarly with the default model for the validation set.

Model 3: Random Forest

The Random Forest model, like the previous one, starts with the default model, which is open with weaker-than-expected results:
Mean accuracy of Random Forest (default) cv: 0.9453
RandomForestClassifier (random_state = 1234)

Parameter Tuning

In the case of parameter scaling, only the normalizer performs positively, which is shown by plot:



With the criterion tuning parameter, you can see that gin performs best, and Tuning Parameter: n_estimators and max_depth show the following results:

	setting	accuracy
0	n_estimator=10_max_depth=3	0.945345
1	n_estimator=10_max_depth=5	0.945345
30	n_estimator=500_max_depth=10	0.945345
31	n_estimator=500_max_depth=30	0.945345
32	n_estimator=500_max_depth=100	0.945345

Evaluation

Although Random forest is one of the most popular machine learning models, it can be stated that it is not the most suitable for this dataset as it has achieved the following values:

Accuracy of CV training set Random Forest(optimal)): 0.9343
Accuracy of Validation set Random Forest(optimal)): 0.9347
F1 Score of Validation set Random Forest(optimal)): 0.9032

Discussion

Evaluation metric	K nearest neighbor	Logistic regression	Random forest
Accuracy	0.9563	0.9945	0.9343
F1 Score	0.9375	0.9375	0.9032

7. Summary

We found that from all datasets, the random forest is the easiest model to work with. It could train quickly and generalize well in all of the data that we have worked with. The only downside is that it has a lot of parameters that we can fine tune the algorithms and we might be at risk of overfitting if we spend too long trying to make it perform better.

Also we found that Logistic regression is more likely to perform poorer than the K-nearest neighbor especially for F1 score. Only in the last dataset, Breast Cancer, does it turn out to perform the best.